

Hardware & Assembly language Programming

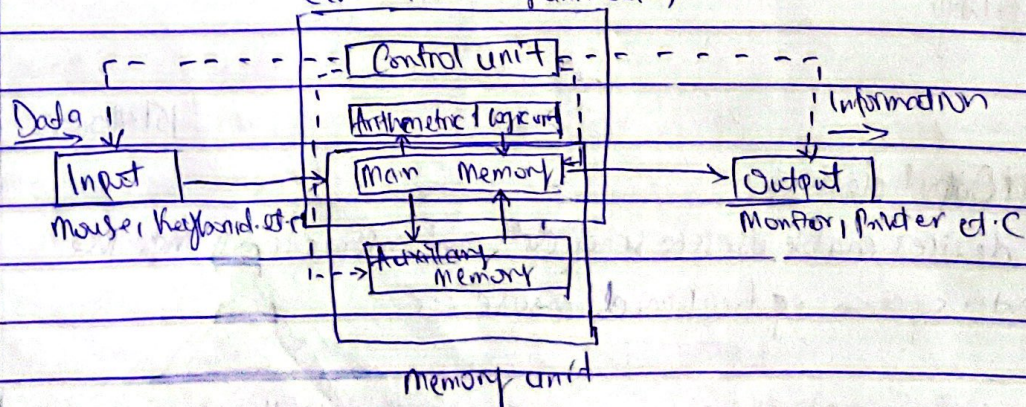
Course outline

- 1) Overview of computer architecture
CPU
- 2) Data representation and addressing modes
- 3) Instruction Set Architecture
- 4) Assembly language programming fundamentals
- 5) Assembly language instructions
- 6) Control flow in assembly language
- 7) Procedures and interrupts
- 8) Input/Output operations in assembly language
- 9) Memory management

Overview of Computer Architecture

Computer architecture refers to the design and organisation of hardware components of a computer system. It encompasses various aspects including the central processing unit (CPU), memory, input and output devices and ^{the} interconnection between these components.

Basic Components of Computer Architecture



CPU

The CPU is often referred to as the brain of the computer. It is responsible for the execution of instructions, performs arithmetic and logic operation and controls the overall operation of the computer.

Components of the CPU includes Arithmetic and logic unit (ALU), Control unit (CU) and registers (main memory)

Factors influencing CPU performance

1. Clock speed
2. Instruction Set Architecture
3. Cache

Memory

Memory stores data and instructions that are currently being processed by the CPU. Types of memory includes; Random access memory and Read only memory.

Other memory hierarchy includes Cache, ~~main memory~~ internal memory, secondary ~~sources~~ storage.

Factors influencing memory design

1. Access time
2. Capacity
3. Volatility

15/4/2025

Input/Output devices

Input devices enable users to send data and instructions into the computer system. eg keyboard, mouse etc

Output devices present the result of a process to the user eg. monitor, printer etc

Secondary storage are storage devices used to store data permanently in a computer system. They come in various types each with its own characteristics, advantages and limitations.

Types of Storage devices

1. Hard disk drive: HDD is a traditional storage device that uses magnetic storage to store and retrieve data. They consist of one or more spinning disk coated in a magnetic material and read/write heads that moves across the surface of the disk to read or write data. They store data permanently in electronic form.
NB: Ram is a semiconductor storage, Hard disk is magnetic in nature.

Advantages

1. It is relatively expensive when compared to Solid state drive
2. High capacity for storing large amount of data
3. Proven reliability and longevity

Disadvantages

1. Slower read/write speed compared to SSD especially for random access
2. Mechanical parts make them more prone to damage from shock, vibration and wear over time.
3. Higher power consumption and noise levels compared to SSD

2. Solid state drive

This uses flash memory to store data electronically without any moving parts. They consist of memory chips that will keep data even when powered off.

Advantages

1. More faster read/write speed compared to Hard disk drive
2. Lower power consumption and noise level due to absence of moving parts
3. It is more durable and resistant to shock and vibration.

Disadvantages

1. Generally more ~~expensive~~ expensive compared to HDD
2. Lower capacity when compared to HDD for the same price
3. Optical drives: It is used in laser ^{technology} ~~technology~~ to read and write data on optical disk
eg. CD, DVD etc. They consist of a spinning disk and a laser assembly that reads and writes data by shining a laser beam onto the disk surface. They are commonly used for distributing software, music etc.

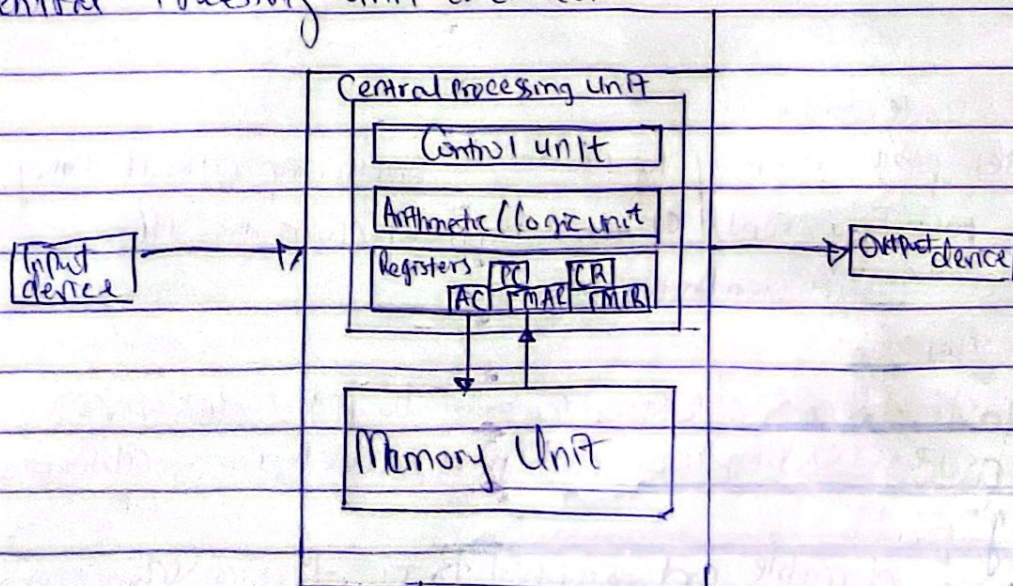
Advantages

1. Portable and ^{convenient} ~~convenient~~ for sharing data or physical media
2. Relatively low cost ^{per disc} ~~per disc~~ compared to HDD and SSD
3. Can store for a very ^{long time} ~~long time~~ under proper storage conditions

Disadvantages

1. Slower read/write speed compared to SSD and HDD
2. Limited storage capacity compared to SSD & HDD
3. Vulnerable to scratches, dust and environmental factors that may degrade its quality

Central Processing Unit architecture



IS - Instruction Set
ISA - Instruction Set Architecture

Motherboard

Types: Single, Dual,

Opcode
 $A \oplus B$ $A \ominus B$
 $C \oplus B$ $C \ominus B$
Operand

Microprocessor helps to speed up the performance of the CPU. It helps to facilitate the performance of the CPU.

- o Opcode
- o Operand

Concept pipelining [Fetch $\rightarrow$ Process $\rightarrow$ Execute]

Instruction Set Architecture defines the machine language interface between the hardware and software.

Components of ISA

- * Instruction Set
- * Registers
- * Data types
- * Memory organisation
- * Address modes

Components of the Motherboard

- ① RAM
- ② Heat sink
- ③ Transformer
- ④ Capacitor
- ⑤ Microprocessor
- ⑥ Fan [cooling system]
- ⑦ Transistor
- ⑧ USB port / serial port
- ⑨ North / south bridge
- ⑩ Jumpers
- ⑪ CMOS battery

Registers are small high speed storage locations within the CPU used to store data intermediate results during computation.

Likely exam questions

10 components of CPU

Diagrammatic

With diagram explain the process of fetch, process and

Execute. / Explain the process of

Types of Registers Instruction Set

- ① Program Counter
- ② Instruction Registers
- ③ General Purpose
- ④ Status Register

16 bits is called a word

Explain the difference
between Program counter
and cache register

- ① Concept of Pipelining ② Clock & Timing ③ ISA

Assignment

① Explain 4 addressing modes with and of a diagram

② And with Systematic drawing show how you can represent data (DSC)

③ Differentiate between Gray code and BCD code
BCD - Binary

30/4/2026

Data representation

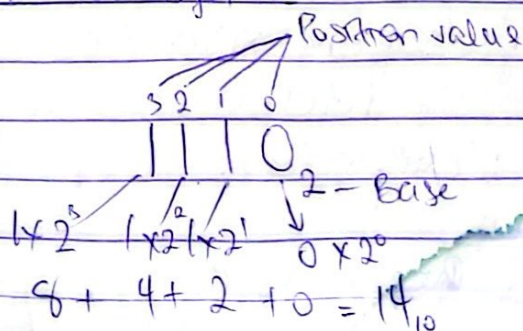
Data is stored ^{represented} in form of 1s and 0s (bits). The fundamental unit of computer storage is a bit and could be ON (1) or OFF (0). A group of 8 bits makes a byte. The processor supports the following data sizes

- 1 Word meaning 2 byte data item (16 bits)
- 2 Double word 4 byte data item (32 bits)
- 3 Quad word 8 byte data item (64 bits)
- 4 Paragraph 16 byte data item (128 bits)

Kilo byte is 1024 bytes

Mega byte is 1048

Number system



$$10101_2$$

$$1 \times 2^4 + 1 \times 2^2 + 1 \times 2^0$$

$$16 + 4 + 1 = 21_{10}$$

Hexadecimal (Base 16)

0=F A=10 B=11 C=12 D=13

E=14 F=15

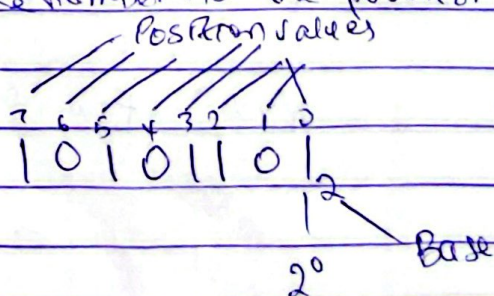
Number system defines the various concepts that are associated with how the computer manipulates data, ex Binary, Decimal, Octa, Hexadecimal and so on.

Key concepts

1. **Base/radix**: Base defines the number of symbols that are allowed in a number system. Starting from 0. example: A base 2 number system allows two symbols 0 and 1, base 3 0, 1 and 2, base 4 0, 1, 2 and 3 and so on.

2. **Position value**: Position value indicates the location of a digit within a given number. It is allocated from the rightmost digit to the leftmost starting with 0, 1, 2 and so on.

3. **Weight/Attached weight**: Every digit within a number has a weight. The weight is determined by raising the base of the number to the position value of the digit in question.



0. **Binary number system**: Also called base 2 number system uses just 2 symbols 0 and 1. 1110

0 Hexadecimal number system: This uses base 16 and the digits ranges from 0-15. By convention the letters A through F are used to represent the hexadecimal digit corresponding to decimal values 10 through 15. The main use of hexadecimal numbers in computing is for abbreviating lengthy binary representation.

Decimal number	Binary representation	Hexadecimal
0	0	0
1	1	1
2	10	2
3	11	3
4	100	4
5	101	5
6	110	6
7	111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Ex: Convert 8CD1 to binary and decimal respectively

Binary

8 C D 1
 $8CD1_{16} = 1000 \quad 1100 \quad 1101 \quad 0001$

$$1000110011010001 = 1 \times 2^{15} + 0 \times 2^{14} + 0 \times 2^{13} + 0 \times 2^{12} + 1 \times 2^{11} + 1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 32768 + 2048 + 1024 + 128 + 64 + 1 = 36049_{10}$$

Hexadecimal

16 | 36049

16 | 2253 R 1

16 | 140 R D

8 R C

0 R 0

$$36049 = 8CD1_{16}$$

Binary arithmetic

Rules

(i) $0 + 1 = 1$

(ii) $0 + 0 = 0$

(iii) $1 + 0 = 1$

(iv) $1 + 1 = 10$

Ex: solve for the following

(a) $1001_2 + 1010_2$

(b) $010_2 + 1100_2$

(c) $1011_2 + 1111_2$

(a) 1001

$+ 1010$

$\underline{10011}_2$

(b) 0010

$+ 1100$

$\underline{1110}_2$

(c) 1011

$+ 1111$

$\underline{11010}_2$

Binary subtraction

1 One's complement

2 Two's complement

1 One's Complement: It simply means opposite. It means converting a digit to its opposite state.

Ex: What is the one's complement of 0001_2 ?

Ans

1110_2

2 Two's Complement: It simply means One's complement plus 1

Ex: Find the 2's complement of 0_2

$$1 + 1 = 10$$

Rules of Binary subtraction

(i) $1011_2 - 1001_2$

(ii) $1001_2 - 1011_2$

$$\begin{array}{r} 1011 \\ - 1001 \\ \hline 0010_2 \end{array}$$

$$\begin{array}{r} 1001 \\ 1011 \\ \hline 0110_2 \end{array}$$

1 Carry out the one complement of the subtrahend

2 Add ~~to~~ the answer to the minuend

3 If there exists an extra bit discarded ~~it~~, the remaining becomes your answer

2 Add 1 to the one's complement

3 Add the 2's complement to the minuend

$$\begin{array}{r} 0101 \\ \hline \end{array}$$

$$\begin{array}{r}
 1 \quad 0110_2 \\
 \underline{1} \\
 0111 \\
 \underline{1011} \\
 \textcircled{1}0010_2
 \end{array}$$

$$\begin{array}{rcl}
 2. & 0100_2 & \text{One's complement} \\
 & + \quad 1 & \text{Two's complement} \\
 & \hline
 & 151 & \\
 & 11001 & \text{Add} \\
 & \hline
 & 1110 & \\
 & 0001 & \text{One's complement} \\
 & \quad 1 & \text{Two's complement} \\
 & \hline
 & - 0010 &
 \end{array}$$

Memory addressing Concepts

7/5/2026

Computer memory is organised as a linear sequence of memory cells each with a unique address. These cells are typically byte addressable meaning each byte in memory has a unique address. Addressing data in memory is a fundamental aspect of programming. This refers to the process of specifying the location of data in computer memory, so that it can be read from or written to by the CPU.

Memory addresses are usually represented as hexadecimal numbers and can range from 0 to the maximum address supported by the system's memory architecture.

Addressing modes

Addressing mode refers to the various ways a processor can access (refer to) data in memory or in registers when

Decoding an instruction.

Common addressing modes

1. Direct addressing: In this, the memory address of operand is exclusively specified in the instructions.
Eg. `MOV AX, [1234h]`
2. Indirect addressing: In this addressing mode the memory address of the operand is specified indirectly through a register or another memory location.
Eg. `MOV AX, [BX]`
3. Indexed addressing: In indexed addressing the memory address of the operand is calculated by adding an offset to a base address stored in a register.
Eg. `MOV AX, [ESI + 10h]`
4. Relative addressing: The address of the operand is calculated relative to the current instruction pointer or program counter.

Instruction set Architecture (ISA)

Instruction set Architecture defines the machine language interface between hardware and software. It specifies the instructions that a processor can ^{execute} ~~execute~~ including their formats, operations, addressing modes and registers.

Components of ISA

1. Instruction set: These are ^{the} sets of instructions supported by the processor each with its opcode and operands.
2. Registers: These are processor's internal storage locations used for data manipulation and temporary storage during execution.
3. Addressing mode: provide technique for specifying operand addresses in instructions such as direct, indirect, relative and indexed addressing.

4. Data types: It provides representations of data supported by the processor including integers, floating point and character data types.

5. Memory organization: It defines the structure of memory including address space, byte ordering and alignment requirements.

Types of instructions

1. Data transfer instruction [mov]

2. Arithmetic and logical instruction

3. Control transfer instructions

4. Input and output instructions

1. Data transfer instructions: helps to move data between registers and memory location

2. Arithmetic and logical instructions: performs arithmetic and logical operations on data

3. Control transfer instructions: Change the sequence of program execution including branching, jumping and sub routine call

4. Input output instructions: Transfers data between the processor and external devices.

Introduction to Assembly language programming

Assignment

1. Describe the basic syntax or structure of Assembly language.

2. Illustrate with an appropriate example.

Assembly language

14/5/2026

Assembly language is a low level programming language that uses mnemonics code to represent machine instructions. It provides a more human readable format for writing programs compared to machine language. It can be used for programming in computer or other programmable device specific to a computer architecture in contrast to high level programming languages, which are generally portable across multiple systems.

Advantages of assembly language

1. It is suitable for time critical jobs.
2. It allows hardware specific complex jobs in an easier way.
3. It requires less memory and execution time.

Structure of an assembly language

An assembly program comprises of three sections and comments.

The sections are as follow

1. **Data section:** The data section is used for declaring initialized data or constants. This data does not change at runtime. You can declare various constant values file names or buffer size etc in this section. The syntax for declaring data section is:
`Section.data`
2. **BSS section:** The BSS section is used for declaring variables. The syntax for declaring is:
`Section.bss`
3. **Text section:** The text section is used for keeping the actual code. This section must begin with the declaration `global main` which tells the compiler where the program execution begins. The syntax for declaring text section is:
`Section.text`
`global main`
`main:`
`add, eax, ebx`

d. Comments: Comments are used to describe elements of a program in assembly language. Comment begins with a semicolon. It may contain any printable character including blank. It can appear on a line by itself or on the same line along with an instruction.

ADD EAX, EBX; This statement add content of EBX to EAX

Assembly language statement or instruction

4. An assembly instruction is a low level programming command written in assembly language and tells the computer's processor to perform a specific task.

Types of instructions

Assembly language program consists of three types of instructions as follows

1. Executable instruction: They tell the processor what to do. Each executable instruction consists of an operation code. Each instruction generates one machine language instruction.
2. Assembly directives / pseudo ops: This tells the assembler about the various aspects of the assembly process. They are non-executable and do not generate machine instructions.
3. Macros: They are a text substitution mechanism.

Syntax of an assembly language instruction

Executable instruction

[label] mnemonic [Operands] [comment]

Assembly language instructions are written one statement per line. Each statement follows the syntax above.

where label is an optional identifier used to mark a specific location in the program.

Mnemonic is the abbreviated form of the instruction operation eg ADD, SUB, MUL, DIV, MOV.

Operands are data or memory addresses on which the instruction operates eg 5, BX, ABX, AX

Types of executable instruction

1 Data transfer instructions (mov): It helps in moving data from source to destination eg MOV syntax MOV, Destination, Source ex MOV AX, BX; move contents of BX into AX,
Instruction Comment

i) MOV AL, 0x1; move immediate value 1 into AL

ii) MOV AX, 5; move immediate value 5 to register AX

iii) MOV BX, [SI]; move data from memory location pointed by SI to register BX

2 Arithmetic & logic instruction

o Arithmetic instruction: Addition

Syntax: ADD destination, source

Ex: ADD AX, BX; $AX = AX + BX$

o Subtraction

Syntax: SUB destination, source

Ex: SUB AX, 5; $AX = AX - 5$

o Multiplication

Syntax: MUL operand

Ex: MUL BX; $AX = AX * BX$

0 Division

Syntax: `Div operand`

Ex: `Div BX`; Divide AX by BX, quotient AX, remainder in DX

Logical Instructions

- Includes AND, OR, XOR, ~~quotient AX~~

Syntax: Mnemonic destination, source

Examples:

1. `AND AX, 0xF0`; perform bitwise AND operation
2. `OR AX, 0xF0`; ✓ ✓ OR ✓
3. `XOR AX, AX`; clear AX ($AX = 0$)

B NOT

Syntax: `Not operand`

Ex: `NOT AX`; perform ~~base~~ bitwise NOT operation on AX

Control flow

It refers to the order in which instructions are executed in a program. In assembly language instructions are executed sequentially unless explicitly ~~ordered~~ ^{altered} by control flow instructions. Control flow allow programs to make decisions like if else logic, repeat action (loop) and jump to different part of the code.

Types of Control flow

1. Branching: It involves changing the normal sequential flow of execution to another part of the program based on a condition or unconditionally.

Unconditional

This allows the program to always jump to specified label or address.

Branching

- Unconditional Jump

Syntax: `JMP LABEL`

`JMP START`; Jump to ~~star~~ `START` label regardless of any condition

Conditional Jump

In conditional jumps the program jumps only if a condition is met

Ex: Zero flag, Carry flag that is it relies on the result of a previous instruction like relational instruction

eg `CMP`

Common Conditional Jump with structures

Instruction	Meaning
<code>JE / JZ</code>	Jump if equal / zero
<code>JNE / JNZ</code>	Jump if not equal / not zero
<code>JG / JNLE</code>	Jump if greater
<code>JL / JNGE</code>	Jump if less

Example

`CMP AX, BX`; Compare `AX` and `BX` equal label

`JE EqualLabel`; Jump to ~~EqualLabel~~ `EqualLabel` if `AX = BX`

2 Looping

It is used when there is need to repeat a block of code multiple times.

Unconditional looping: This increases the `CX` register and jumps to a label if `CX  $\neq$  0`

Syntax

LOOP LABEL

Example: MOV CX, 5
 LOOP LABEL
 ; code to repeat
 LOOP LABEL; Repeat until CX = 0

Conditional looping

These combines conditional jumps with logic

Example

```
LOOP_START  
COMP AX, 0  
JE END_LOOP  
DEC AX  
JMP LOOP_START  
END_LOOP
```

Note: Procedures / subroutines, Parameter passing in assembly language